AI FitTrack

Project Documentation

API Documentation, Project Features, Testing and Implementation Overview

Node.js | Express | MongoDB | JWT Authentication | Gemini AI | Thunder Client

Team ID: SWTID-2026-7917

Role	Name	Email
Team Leader	RAMOLIYA Y	24csramoliya.y075@gmail.com
Team Member 1	Rajeswari K	24csrajeswarik074@gmail.com
Team Member 2	Priya B	24cspriya71@gmail.com
Team Member 3	Priyadharshini P	24cspriyadarshini072@gmail.com
Team Member 4	Priyanka S	24cspriyanka.s073@gmail.com

1. Introduction

AI FitTrack is a fitness management and AI-assisted workout platform designed to help users manage their fitness activities and receive personalized workout recommendations and fitness insights. The project provides a REST API backend built with Node.js and Express, stores data in MongoDB, protects user-specific resources with JSON Web Tokens (JWT) and connects to the Gemini AI service to generate intelligent responses.

This document describes the API, the main features, the authentication flow, the testing procedure followed in Thunder Client for Visual Studio Code, the results observed during testing, and the planned next steps. It is intended for the project team, evaluators and developers who will later build a web or mobile client on top of the API.

Scope of this document

The scope of this version is the backend API: user authentication, workout management, the two AI routes and their testing in Thunder Client. A web or mobile client, cloud deployment and automated test suites are not part of this version and are listed under future enhancements.

Technology stack

Technology	Role in the project
Node.js	JavaScript runtime that executes the backend server.
Express	Web framework used for routing, middleware and request handling.
MongoDB	Database used for persistent storage of user and workout information.
JSON Web Token (JWT)	Signed token used for stateless authentication of protected routes.
Gemini AI	Generative AI service used for workout recommendations and fitness insights.
Thunder Client	Visual Studio Code extension used to send requests and inspect responses during testing.

2. Project Objectives

- **Simple and secure API** – provide a clean fitness management API that is easy to understand and safe to use.

- **Secure account access** – allow users to register and log in using credentials and receive a token.
- **Workout management** – manage workout information through standard REST operations.
- **AI workout recommendations** – generate personalized workout suggestions with the Gemini AI service.
- **AI fitness insights** – generate meaningful observations about a user's fitness information.
- **Protected resources** – use JWT authentication to guard user-specific and AI-related routes.
- **Clear responses** – return consistent JSON responses with suitable HTTP status codes and error messages.

3. Key Features

AI FitTrack combines conventional fitness data management with AI-generated guidance. The main features implemented in the backend are summarised below.

Feature	Description
User registration and login	New users create an account through /api/auth/register; existing users authenticate through /api/auth/login.
JWT-based authentication	A successful login returns a signed token. The token is sent as a Bearer token and verified before protected routes run.
Workout management	Workout records can be created, retrieved, updated, partially updated and deleted through the /api/workouts routes.
AI workout recommendations	Authenticated users can request a personalised workout suggestion generated by the Gemini AI service.
AI fitness insights	Authenticated users can request AI-generated insights based on their fitness information.
MongoDB connectivity	User and workout information is stored persistently in a MongoDB database.
RESTful API architecture	Resource-based URLs, standard HTTP methods and JSON payloads keep the API predictable.
Centralized error handling	Errors are handled in one place so that clients receive a consistent status code and message.
Thunder Client testing	Every endpoint can be tested directly inside Visual Studio Code without an external tool.

4. System Overview and Architecture

The system follows a client-server architecture. A client, such as a web or mobile application, sends HTTP requests to the AI FitTrack backend. The backend processes each request, checks authentication when required, communicates with the database, and returns a JSON response. AI-related requests are additionally forwarded to the configured Gemini AI service.

Client	Routes and Middleware	Controllers and Services	Data and AI Layer
Web or mobile app; Thunder Client during testing	Route matching and JWT verification for protected routes	Business logic, input validation and AI request handling	MongoDB for users and workouts; Gemini AI for recommendations and insights

Request lifecycle

- The client sends an HTTP request with a JSON body and, for protected routes, an Authorization header.
- The router matches the URL and method to the correct route.
- Authentication middleware verifies the JWT. Requests without a valid token stop here with 401 Unauthorized.
- The controller and service layer validate the input, read or write MongoDB or call Gemini AI, and return a JSONresponse. Errors go to the centralized error handler.

The code base is organised by routes, controllers, middleware and services, which keeps each responsibility separate and makes the backend easier to maintain and extend.

Responsibilities of each layer

- **Routes** – map each URL and HTTP method to the correct controller and attach middleware where protection is required.
- **Middleware** – verifies the JWT on protected routes and passes only valid requests to the next handler.
- **Controllers** – read the request, validate the input, perform the required database or AI operation and build the response.
- **Services** – contain the logic that communicates with external systems, such as the Gemini AI service.
- **Error handler** – catches errors from every layer and returns a consistent status code and JSON message.

5. API Base URL and HTTP Methods

During local development the backend runs on port 5000. All endpoints in this document are relative to the following base URL.

```
Base URL: http://localhost:5000
```

When the backend is deployed, only the base URL changes; the route paths remain the same. The project uses five HTTP methods.

Method	Purpose	Example use
GET	Retrieve data	Get workouts, user information, fitness data
POST	Create or submit data	Register, login, request an AI recommendation
PUT	Update existing data	Update workout or user-related information
PATCH	Partially update data	Update selected fields of a workout or profile
DELETE	Remove data	Delete a workout or other removable resource

The exact PUT, PATCH and DELETE endpoints depend on the workout and controller routes implemented in the project. They should be documented with their final route paths once those routes are confirmed.

Running the backend locally

- Install Node.js and make sure a MongoDB database is available.
- Install the project dependencies with npm install in the project root folder.
- Create the .env file with the MongoDB connection string, the JWT secret and the Gemini API key.
- Start the server and confirm that the terminal reports a successful MongoDB connection and that the server is listening on port 5000.

The health check request described in Section 9 can then be used to confirm that the API is reachable before the other endpoints are tested.

6. API Endpoint Documentation

Method	Endpoint	Auth	Purpose
GET	/	No	Check whether the API is running
POST	/api/auth/register	No	Create a new user account
POST	/api/auth/login	No	Authenticate a user and obtain a JWT
POST	/api/ai/workout-recommendation	Yes	Generate an AI workout recommendation
POST	/api/ai/fitness-insights	Yes	Generate AI fitness insights

Method	Endpoint	Auth	Purpose
GET	/api/workouts/...	Usually yes	Retrieve workout data; final path depends on workout routes
POST	/api/workouts/...	Usually yes	Create workout data; final path depends on workout routes
PUT	/api/workouts/...	Usually yes	Update an existing workout; final path depends on workout routes
PATCH	/api/workouts/...	Usually yes	Partially update a workout; final path depends on workout routes
DELETE	/api/workouts/...	Usually yes	Delete an existing workout; final path depends on workout routes

Public endpoints are the health check, registration and login. Protected endpoints are the two AI routes and, in general, the workout routes, because they work with user-specific data. Protected endpoints require a valid JWT Bearer token, as explained in the next section.

7. Authentication and Authorization

AI FitTrack uses stateless JWT authentication. When a user logs in successfully, the server signs a token and returns it in the login response. The client stores this token and attaches it to every later request that targets a protected resource. The server verifies the token on each request, so no session data needs to be kept on the server.

Obtaining and using a token

- Register a user using the registration endpoint.
- Log in using the login endpoint.
- Copy the JWT returned in the login response.
- In Thunder Client, open the Auth tab and select Bearer Token.
- Paste the JWT into the token field.
- Send the protected API request.

Behind the scenes, Thunder Client sends the token in the standard authorization header:

```
Authorization: Bearer <JWT token>
```

Authorization outcomes

Situation	Status	Explanation
No token supplied	401	The request reaches a protected route without an Authorization header.
Invalid or expired token	401	The token cannot be verified, so the user must log in again.
Valid token, no permission	403	The user is authenticated but not allowed to access the resource.
Valid token	200 / 201	The request is processed and a successful response is returned.

Token expiry and re-login

Because JWT authentication is stateless, a token remains valid until it expires. When a token expires, the user logs in again and receives a new one; no server-side session store is required. The secret used to sign tokens must be kept private, because anyone who knows it could create valid tokens.

8. Request and Response Format

The API uses JSON for both requests and responses. Requests that contain a body should send the Content-Type: application/json header. A successful response generally contains a success indicator and the requested result; error responses contain an appropriate HTTP status and an error message.

Example: login request

```
POST http://localhost:5000/api/auth/login

{
  "email": "test@example.com",
  "password": "123456"
}
```

Example: login response (200 OK)

```
{
  "success": true,
  "token": "<JWT token>",
  "user": { ... }
}
```

Example: protected AI request

```
POST http://localhost:5000/api/ai/workout-recommendation
Authorization: Bearer <JWT token>
Content-Type: application/json
```

The JSON body of an AI request carries the fitness details expected by the AI controller. The route responds only when the Bearer token is valid.

Example: error response (401 Unauthorized)

```
{
  "success": false,
  "message": "No token provided"
}
```

Error responses use the same JSON style as successful responses, so a client can read the status code and the message in one place. The wording above is illustrative; the exact text is produced by the authentication middleware.

Important HTTP status codes

Status	Meaning	Typical situation
200	OK - Request successful	Successful GET, login, update or AI response
201	Created - Resource created	Successful registration or creation
400	Bad Request - Invalid request	Missing or invalid input
401	Unauthorized - Authentication required or failed	Missing or invalid JWT
403	Forbidden - Access denied	Authenticated user lacks permission
404	Not Found - Endpoint or resource not found	Invalid URL or missing resource
500	Internal Server Error - Server-side failure	Unexpected backend error

9. Thunder Client Testing Procedure

The API was tested with the Thunder Client extension in Visual Studio Code. The following sequence was used so that each stage builds on the previous one.

- Start the backend server and confirm that it is listening on port 5000.
- Open Thunder Client in VS Code.

- Test GET http://localhost:5000/ and confirm a 200 OK response.
- Test user registration.
- Test user login and save the JWT token.
- Use the JWT as a Bearer Token for protected requests.
- Test workout GET, POST, PUT, PATCH and DELETE operations according to the implemented workout routes.
- Test the AI workout recommendation endpoint.
- Test the AI fitness insights endpoint.
- Check the status code and JSON body of every request.

Tip: the token can be stored in a Thunder Client environment variable (Env tab) so it does not have to be pasted again after every login.

Test case summary

Test	Endpoint	Expected	Observed
Health check	GET /	200 OK	200 OK (41 ms)
Login	POST /api/auth/login	200 OK with a JWT	200 OK (81 ms)
Registration with invalid data	POST /api/auth/register	400 Bad Request	400 shown in the terminal log
AI request without a token	POST /api/ai/workout-recommendation	401 Unauthorized	401 with a "no token provided" message
AI requests with a valid token	POST /api/ai/...	200 OK	To be verified after the Gemini key is configured
Workout PUT, PATCH and DELETE	/api/workouts/...	200 OK	To be verified against the final workout routes

10. Testing Results and Screenshots

10.1 Authentication test

Figure 1 shows the login request in Thunder Client together with the server terminal. The request was sent with a JSON body containing an email and a password.

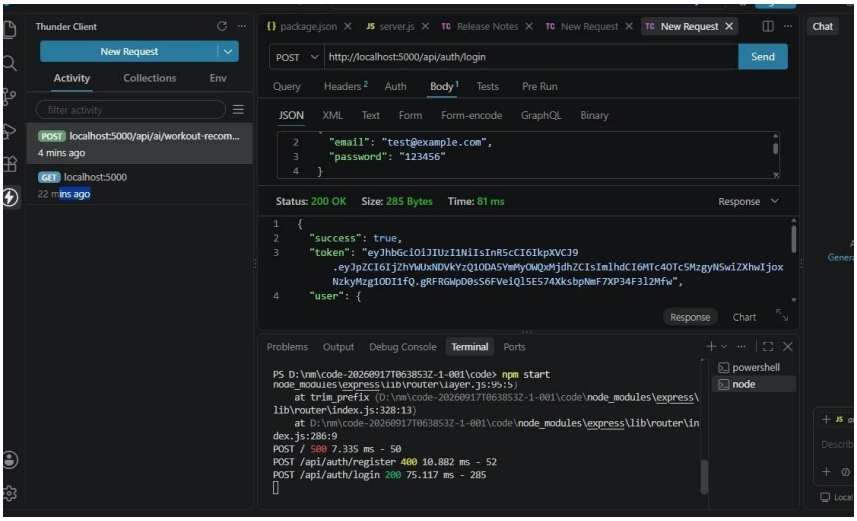


Figure 1. Login request returning 200 OK with a JWT (Thunder Client and VS Code terminal)

- **Request and result:** POST /api/auth/login with an email and password returned 200 OK (285 bytes, 81 ms). The body contains "success": true, a token and a user object.
- **Terminal log:** the server logged POST /api/auth/login 200 75.117 ms - 285, confirming the result on the backend.
- **Token use:** the returned JWT must be sent as a Bearer token on protected requests.

10.2 Health check test

Figure 2 shows the health check, which confirms that the server is running and reachable.

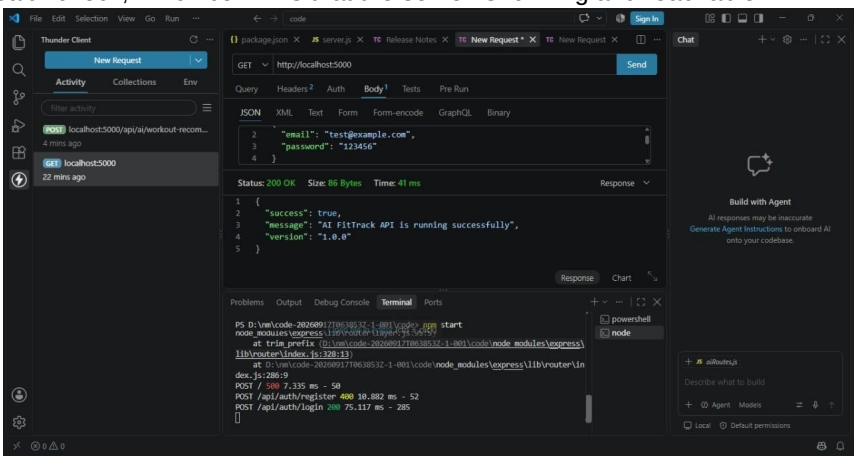


Figure 2. Health check GET http://localhost:5000 returning 200 OK

- **Request and result:** GET http://localhost:5000 with no authentication returned 200 OK (86 bytes, 41 ms).
- **Response body:** success is true, the message states that the AI FitTrack API is running successfully, and the version is 1.0.0.

10.3 Summary of test evidence

Both screenshots show successful responses returned quickly: the login request completed in 81 ms and the health check in 41 ms, which is suitable for local development. The terminal log matches the Thunder Client results, confirming that each request reached the backend and was processed as expected.

Screenshots of the AI endpoints and of the workout operations can be added to this section once the Gemini key is configured and the final workout routes have been tested.

11. Issues and Observations During Testing

Observation	Explanation	Resolution
AI recommendation request returned 401 with a "no token provided" message	The AI route is protected and the request was sent without a JWT. This showed that JWT protection is active.	Log in, copy the token and send it as a Bearer token.
Startup warning about the Gemini API key	The Gemini service needs a valid key in the project's environment configuration.	Configure the key in the environment file and restart the server (see Section 12).
Terminal shows POST / returning 500	The health check is available with GET. A request with a different method appears to have reached the error handler.	Use GET for the health check; consider returning 404 for unmatched routes and methods.
Terminal shows POST /api/auth/register returning 400	The registration request was rejected as a bad request, for example because of invalid input or an email that is already registered.	Check the response message and use valid, unique details.

12. Gemini AI Configuration

During server startup, a warning indicated that the Gemini API key must be configured when using the Gemini API. AI functionality should therefore be tested only after the project's expected environment variable has been set to a valid key.

- Create or open the environment file (.env) in the project root.
- Add the valid Gemini API key under the variable name expected by the project.
- Restart the server, confirm the startup warning no longer appears, then log in and retest both AI endpoints withthe Bearer token.

For security, the actual API key must never be included in project documentation, screenshots, source control or shared videos.

Checklist before testing the AI routes

- The .env file exists and contains a valid Gemini API key.
- The server was restarted after the environment file was changed.
- The startup warning about the Gemini key no longer appears in the terminal.
- A fresh JWT was obtained from the login endpoint and set as the Bearer Token.
- The request body contains the fitness details expected by the AI controller.

13. Database

The project uses MongoDB for persistent data storage. During testing, the terminal showed that MongoDB connected successfully when the server started. The database stores user and workout-related information according to the models implemented in the project. The database connection string should be kept in an environment variable, and user passwords should be stored in hashed form rather than as plain text.

Data handling practices

- Passwords are hashed before they are saved; plain-text passwords are never stored or returned in a response.
- Each email address should be unique, so a second registration with the same email is rejected as a bad request.
- Workout records should be linked to the user who created them, so users work only with their own data.
- The connection string and other credentials are read from the environment file and not written in the source code.

14. Security Considerations

Practice	Description
JWT authentication	Protect user-specific and AI-related resources with a valid token.
Environment variables	Keep API keys and database credentials outside the source code.
Secret files	Do not commit .env files containing secrets to source control.
Validation and status codes	Validate incoming request data and return appropriate HTTP status codes for every outcome.
Data exposure	Avoid exposing sensitive user information in API responses.
HTTPS	Use HTTPS when the application is deployed publicly.

Additional safeguards recommended

- **Rate limiting** – apply limits to the login and AI routes to reduce repeated login attempts and control Gemini usage.
- **CORS policy** – allow only the known client origin once a web or mobile client is built.
- **Token lifetime** – use a reasonably short expiry so that a lost token is useful for only a limited time.
- **Dependency checks** – run npm audit regularly and update packages that have known vulnerabilities.

15. Functional and Non-Functional Requirements

Functional requirements

Requirement	Description
User Management	Users can register and authenticate.
Authentication	Protected resources require valid authentication.
Workout Management	Users can create, retrieve, update and delete workout information where routes are implemented.
AI Recommendation	The system can provide AI-generated workout recommendations.
Fitness Insights	The system can provide AI-generated fitness insights.
Database	User and fitness information can be stored in MongoDB.
API Testing	Endpoints can be tested using Thunder Client.

Non-functional requirements

The API should provide clear JSON responses and handle invalid requests gracefully. Protected resources should require authentication. The backend should be maintainable and organised by routes, controllers, middleware and services, and secrets should be managed securely through environment configuration.

16. Project Status Based on Testing

Component	Status	Observation
Server	Working	Backend successfully started on port 5000.
MongoDB	Working	MongoDB connection was shown in the terminal.
Health Check	Working	GET / returned 200 OK.
Authentication	Working	Login returned 200 OK and a JWT.
JWT Protection	Working	Protected AI request returned 401 when no token was supplied.
AI Routes	Implemented; needs final functional verification	Routes are present; the final AI response depends on authentication and Gemini configuration.
PUT / PATCH / DELETE	Route-dependent	Should be tested against the final workout routes implemented in the project.

17. Project Demonstration Video

A project demonstration video was recorded during development. The video is an MP4 file of approximately 163 MB. To transfer it from a laptop to a phone, WhatsApp, Nearby Sharing, OneDrive and USB were considered; Nearby Sharing or OneDrive are suitable for a file of this size. Before the video is shared, make sure that no API keys or credentials are visible in it.

18. Future Enhancements

Feature enhancements	Technical enhancements
Frontend dashboard for users	Deploy the backend and database to a cloud environment
Workout history and progress tracking	Add automated API tests
BMI, calorie and activity tracking	Add role-based access if administrators are required
Personalised nutrition recommendations	Improve AI prompt personalisation using user fitness data
Charts for fitness progress	Return 404 for unmatched routes through the centralized error handler

The first priority is to finish verifying the AI routes and the workout operations, followed by automated API tests. The frontend dashboard, progress charts and nutrition recommendations can then be built on top of a stable and tested API.

19. Quick API Reference

Operation	Method	Local URL
Health check	GET	http://localhost:5000/
Register	POST	http://localhost:5000/api/auth/register
Login	POST	http://localhost:5000/api/auth/login
Workout list / details	GET	http://localhost:5000/api/workouts/...
Create workout	POST	http://localhost:5000/api/workouts/...
Update workout	PUT	http://localhost:5000/api/workouts/...
Partial update	PATCH	http://localhost:5000/api/workouts/...
Delete workout	DELETE	http://localhost:5000/api/workouts/...
AI workout recommendation	POST	http://localhost:5000/api/ai/workout-recommendation
AI fitness insights	POST	http://localhost:5000/api/ai/fitness-insights

20. Conclusion

AI FitTrack provides a REST-based backend for authentication, workout management and AI-assisted fitness functionality. The project has been tested through Thunder Client, including the health check, authentication and protected AI routes. The next validation stage is to complete the Gemini configuration and verify all implemented workout GET, POST, PUT, PATCH and DELETE operations with valid test data.

The tested parts of the system meet the main objectives: the server starts, MongoDB connects, users can log in and receive a token, and JWT protection blocks unauthenticated access to the AI routes. With the remaining verification and the enhancements listed in Section 18, AI FitTrack can grow from a tested backend into a complete fitness application.

Resources

API testing:

Code:https://drive.google.com/drive/folders/12TylBxjUlta97W2Q5tZYwrJro1zmlerl

Demovideo:https://drive.google.com/file/d/19EN0e32B5hfU2dl7xT13tNtzghOswAl8/view?usp=drivesdk